

SOMMAIRE

DEDICACE.....	ii
REMERCIEMENTS	iii
RESUME.....	iv
ABSTRACT	v
TABLES DES FIGURES	vi
INTRODUCTION GENERALE.....	1
CHAPITRE I : ETAT DE L'ART.....	2
CHAPITRE II : ANALYSE DES BESOINS ET SPÉCIFICATIONS	14
CHAPITRE III : CONCEPTION DE L'ARCHITECTURE	19
CHAPITRE IV : DÉPLOIEMENT ET MISE EN ŒUVRE.....	34
CHAPITRE V : TESTS ET VALIDATION	40
CONCLUSION GENERALE	49

DEDICACE

A

Mon cousin Jean Nestor YAPENDE qui a été toujours là pour moi, et qui m'a donné un magnifique modèle de labeur et de persévérance. J'espère qu'il trouvera dans ce travail toute ma reconnaissance et tout mon amour.

A mes chers frères et adorable sœur.

A ma mère YAPENDE Tatiana et mon père POUTIA.

A mon binôme SELEDONO Stéphane et sa respectueuse famille.

A mes ami(e)s.

REMERCIEMENTS

La réalisation de ce travail n'aurait pas été possible sans l'appui et les contributions de nombreuses personnes, auxquelles nous adressons ici notre profonde gratitude.

- Nous tenons à exprimer notre profonde gratitude à **Monsieur Thierry TOYEN**, Enseignant-Chercheur à l'Université de Bangui et Ingénieur en Réseaux et Télécommunications, pour son accompagnement, sa disponibilité et ses précieux conseil tout au long de la réalisation de ce travail ;
- Nos sincères remerciements s'adressent également au **Professeur Jean M'BOLIGUIPA**, Directeur de l'Institut Supérieur de Technologie, pour son sérieux et son engagement constant dans le suivi des programmes académiques au sein de notre Institut ;
- Nous remercions le **Professeur Noé Landry Privace M'BOUANA**, Directeur des Etudes de l'Institut Supérieur de Technologie, pour son encadrement et sa contribution à la qualité de notre formation ;
- Notre reconnaissance va aussi au **Docteur Martyrs OULOIN**, Chef du département de Génie Industriel pour la rigueur et la discipline avec lesquelles il dirige notre département ;
- Nous exprimons notre gratitude à l'ensemble du **corps professoral et administratif** de l'Institut Supérieur de Technologie, qui a largement contribué à notre formation et continue d'œuvrer avec dévouement pour l'encadrement des générations futures.
- Nous remercions également **nos collègues de promotion**, pour leur soutien, leur collaboration et leur esprit d'entraide durant les trois années de notre cursus ainsi que lors de l'élaboration de ce travail ;

Enfin, nos sincères remerciements vont à nos **parents, amis et connaissances** pour leur soutien moral et matériel indéfectible, ainsi qu'à toutes personnes qui ont contribué de près ou de loin à la réalisation de ce travail et dont les noms ne n'ont pas été cités ici. Qu'elles trouvent l'expression de notre profonde reconnaissance.

RESUME

L'objectif de notre projet était de mener une étude de la technologie WebRTC afin d'installer et de configurer un serveur permettant la communication entre deux ordinateurs via Internet.

Pour répondre au cahier des charges que nous nous étions fixés, nous avons identifié et analysé les différents outils nécessaires au bon fonctionnement du service.

La technologie WebRTC devait être hébergée par un serveur Asterisk, tandis que les clients utiliseraient une page web pour se connecter au serveur et passer des appels via des clients WebRTC SIPML5. L'étude de ces outils nous a permis de constater la nécessité de deux services supplémentaires : les serveurs STUN et TURN.

Une fois le fonctionnement de ces différents composants compris, nous avons procédé à l'installation des premiers éléments et aux tests de configurations spécifiques. Le premier élément installé fut le serveur Asterisk, qui constitue la base de toute l'infrastructure. Après sa configuration et la création des comptes utilisateurs, nous nous sommes intéressés au client SIPML5. En renseignant correctement les champs requis, nous avons réussi à connecter des utilisateurs sur différents ordinateurs. Nous avons ensuite effectué un premier appel audio, suivi d'un appel vidéo.

ABSTRACT

The objective of our project was to conduct a study of WebRTC technology in order to install and configure a server that enables communication between two computers over the internet.

To meet the specifications, we set for ourselves, we identified and analyzed the various tools required for the service to function. The WebRTC technology was to be hosted by an **Asterisk server**, while clients would use a web page to connect to the server and place calls via **SIPML5 WebRTC clients**. Our study of these tools revealed the necessity of two additional services: **STUN and TURN servers**.

Once we understood how these different components work, we began installing the initial elements and testing specific configurations. The first of these was the Asterisk server, which serves as the foundation of the entire infrastructure. After configuring it and creating user accounts, we focused on the SIPML5 client. By correctly filling in the required fields, we successfully connected users across different computers. We then proceeded to conduct an initial audio call, followed by a video call.

TABLES DES FIGURES

Figure 1 : Google Meet

Figure 2 : Microsoft Teams

Figure 3 : Zoom

Figure 4 : BigBlueButton

Figure 5 : WebRTC

Figure 6 : Les fonctionnalités du WebRTC

Figure 7 : Types d'architecture de conférence WebRTC

Figure 8 : Asterisk

Figure 9 : Architecture

Figure 10 : Schéma conceptuel

Figure 11 : Protocole SIP

Figure 12 : Protocole H.323

Figure 13 : NAT traversal (STUN/TURN)

Figure 14 : Page d'accueil de sipml5

Figure 15 : Mode expert

Figure 16 : Formulaire d'enregistrement

Figure 17 : Call control

Figure 18 : Test audio

Figure 19 : Tests vidéo

Figure 20: Latence

INTRODUCTION GENERALE

La visioconférence occupe aujourd'hui une place centrale dans les environnements professionnels, éducatifs et collaboratifs, notamment dans un contexte marqué par une digitalisation accrue des activités et une mobilité grandissante des acteurs. Face à ces évolutions, les organisations recherchent des solutions de communication en temps réel à la fois performantes, sécurisées et interopérables. Cela rend indispensable la conception d'architectures techniques robustes, évolutives et capables de répondre à la diversité des besoins opérationnels. La présente étude s'inscrit dans cette perspective en analysant et en mettant en œuvre une architecture de visioconférence fondée sur WebRTC, technologie open source de référence pour les communications audio-vidéo en temps réel, et sur Asterisk, serveur de téléphonie IP largement utilisé dans les systèmes de communication unifiée. L'association de ces deux technologies constitue une approche mature et flexible pour la construction de solutions collaboratives modernes.

L'objectif principal de ce travail consiste à proposer une architecture de visioconférence intégrant WebRTC et Asterisk, conçue pour offrir une solution fiable, adaptable et conforme aux exigences contemporaines en matière de communication en temps réel.

Ainsi, la problématique suivante se dégage : **Comment concevoir et déployer une architecture de visioconférence efficace fondée sur WebRTC et Asterisk ?**

Afin d'apporter des éléments de réponse à cette problématique, ce travail est structuré en cinq chapitres complémentaires.

Le premier chapitre présente un état de l'art des technologies de visioconférence existantes ainsi qu'un panorama des solutions propriétaires du marché. Le deuxième chapitre est consacré à l'analyse des besoins et à la spécification fonctionnelle et technique du système. Le troisième chapitre développe la conception de l'architecture proposée, en détaillant ses composants et leurs interactions.

Le quatrième chapitre décrit le processus de déploiement et la mise en œuvre opérationnelle de la solution.

Enfin, le cinquième chapitre est dédié aux tests, à l'évaluation des performances et à la validation du système finalisé.

CHAPITRE I : ETAT DE L'ART

Contrairement à la visiophonie, la visioconférence permet de faire dialoguer plus que deux personnes simultanément et donc de simuler une conférence à plusieurs lorsque tous les membres ou une partie d'entre eux sont éloignés géographiquement. Pour arriver à cela, il existe plusieurs façons de s'organiser. Ce que nous détaillerons dans ce chapitre. Nous allons voir toutes les technologies de visioconférence existantes, la présentation du WebRTC, ainsi que la présentation de l'Asterisk.

I.1 Technologies de visioconférence existantes

La visioconférence repose sur ensemble de technologie permettant la transmission en temps réel de la voix, de la vidéo et des données entre plusieurs utilisateurs distants. Ces technologies peuvent classées en solutions propriétaires, solution open source et technologique.

I.1.1 Solutions propriétaires

La solution propriétaire sont développées et maintenues par des entreprises privées.

Elles offrent généralement une grande stabilité, une interface conviviale et des services intégrés, mais leur code source n'est pas accessible.

I.1.1.1 Google Meet

Google Meet est une application de visioconférence de Google qui permet aux utilisateurs de se réunir en ligne via des appels vidéo, que ce soit pour des réunions professionnelles, des cours ou des interactions personnelles. Il est autrefois connu sous le nom de Google Hangouts, est intégré à la suite Google Workspace, mais est aussi accessible gratuitement avec un compte Google. Meet offre des fonctionnalités comme le partage d'écran, l'enregistrement des réunions et la participation via un simple lien, rendant la collaboration à distance facile et flexible depuis n'importe quel appareil.

Fonctionnalité clés de Google Meet

-  Appel vidéo sécurisés : Permet des visioconférences sécurisées pour communiquer avec des collègues ou des proches.
-  Partage d'écran : Les participants peuvent partager leur écran pour présenter des documents ou des présentations.
-  Enregistrement de réunions : La possibilité d'enregistrer des appels vidéo pour une consultation ultérieure.

- ✚ Accès simple : Les réunions peuvent être rejointes via un lien partagé, un numéro de téléphone, ou directement depuis l'application.
- ✚ Intégration à Google Agenda : Permet de planifier et synchroniser les réunions avec le calendrier Goog.

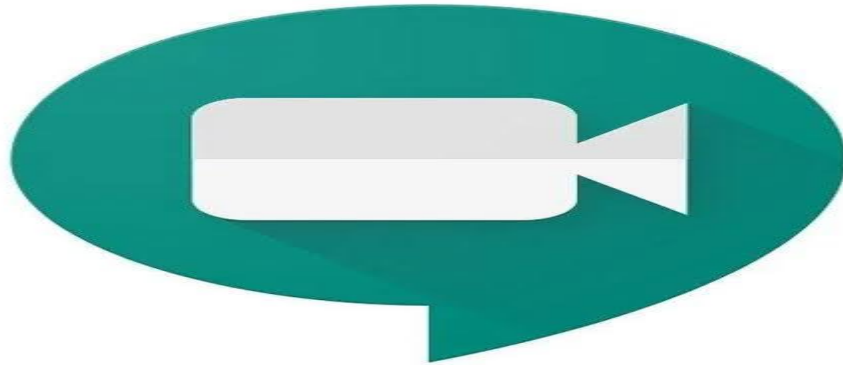


Figure1: Google Meet (*Source : Google*)

I.1.1.2 Microsoft Teams

Microsoft Teams est une plateforme de collaboration conçue par Microsoft pour faciliter la communication et le travail en équipe au sein des entreprises. Elle permet de centraliser les conversations (chat), les réunions virtuelles (visioconférence), le partage de fichiers et l'intégration d'applications dans un seul espace de travail unifié. En tant que successeur de skype, Teams offre une suite plus intelligente et polyvalente pour organiser le travail, maintenir les équipes connectées et atteindre les objectifs professionnels plus efficacement.



Figure 2 : Microsoft Teams (*Source: Google*)

I.1.1.3 Zoom

Zoom est une plateforme propriétaire de visioconférence basée sur le cloud, très populaire, permettant des réunions vidéo, audio, des webinaires et de la collaboration via partage d'écran, chat, tableau blanc, et enregistrement, accessible sur PC, mobile et tablette, offrant des versions gratuites avec limitations (ex : 40 min) et des plans payants pour plus de participants (jusqu'à 1000+) et de fonctionnalités (Zoom Phone, Rooms, Events). Sa facilité d'utilisation, haute qualité vidéo/audio HD et fonctionnalités de gestion (mute, lever la main) en ont fait un standard, malgré des préoccupations passées de sécurité.



Figure 3 : Zoom (*Source Google*)

Avantage des solutions propriétaires :

- ✚ Facilité d'utilisation

- + Qualité audio/vidéo optimisée
- + Support technique professionnel

Inconvénients

- + Coût élevé pour les versions avancées
- + Dépendance au fournisseur
- + Faible flexibilité de personnalisation

I.1.2 Solution Open source

Les solutions open source offrent une plus grande liberté de configuration et sont particulièrement adaptées aux environnements de petites et moyennes entreprises, académiques et aux projets de recherche.

I.1.2.1 BigBlueButton

BigBlueButton est une application Web basé sur HTML5, contrairement à de nombreux systèmes de conférence Web commerciaux qui nécessitent l'installation de logiciels, BigBlueButton s'exécute dans votre navigateur Web. Vous cliquez sur le lien (comme dans Greenlign), votre navigateur exécute BigBlueButton et vous invite à rejoindre le pont audio. Il n'y a aucun plugin à télécharger, aucun logiciel à installer. BigBlueButton fournit le partage audio, vidéo et d'écran de haute qualité à l'aide de la prise en charge intégrée du navigateur pour les bibliothèques de communication Web en temps réel (WebRTC).

Le nom de BigBlueButton (« Gros bouton bleu ») vient de la première idée des concepteurs selon laquelle une visioconférence sur le Web doit être aussi simple à démarrer qu'en appuyant sur un gros bouton bleu, BigBlueButton offert dans dans 21 langues. Il est écrit en HTML5 et JavaScript, et livré sous licence open source. Installé sur un serveur distant sous Ubuntu il peut être également téléchargé comme machine virtuelle.

BigBlueButton : Spécialisé pour l'éducation et la formation en ligne, auto-hébergeable, avec tableau blanc et fonctions de sondage.



Figure 4 : BigBlueButton (*Source : Google*)

I.1.2.2 Présentation du WebRTC

WebRTC qui veut dire (web Real-Time communication) est un ensemble de standard et API permettant des communications audio, vidéo et des données en temps réel entre navigateurs (ou entre navigateur et serveur) sans plugins externes. Il comprend les éléments fondamentaux pour des communications de haute qualité sur le web, tels que les composants réseaux, audio et vidéos utilisés dans les applications de conversation vocale et vidéo.

WebRTC est une technologie permettant les communications directes (sans passer par un serveur web) et en temps réel entre plusieurs navigatrices soutenues par Google, Mozilla et Opera au sein des standards du World Wide Web consortium (W3C), dont les premières ébauches sont apparues en mai 2011.



Figure 5 : WebRTC (*Source : Google*)

I.1.2.2.1 Les fonctionnalités du WebRTC

La technologie WebRTC offre plusieurs fonctionnalités, voici quelques fonctionnalités les plus courantes :

- ✚ Appel audio
- ✚ Appel vidéo
- ✚ Messagerie instantanée
- ✚ Transfert de fichier
- ✚ Transfert d'appel
- ✚ Partage d'écran

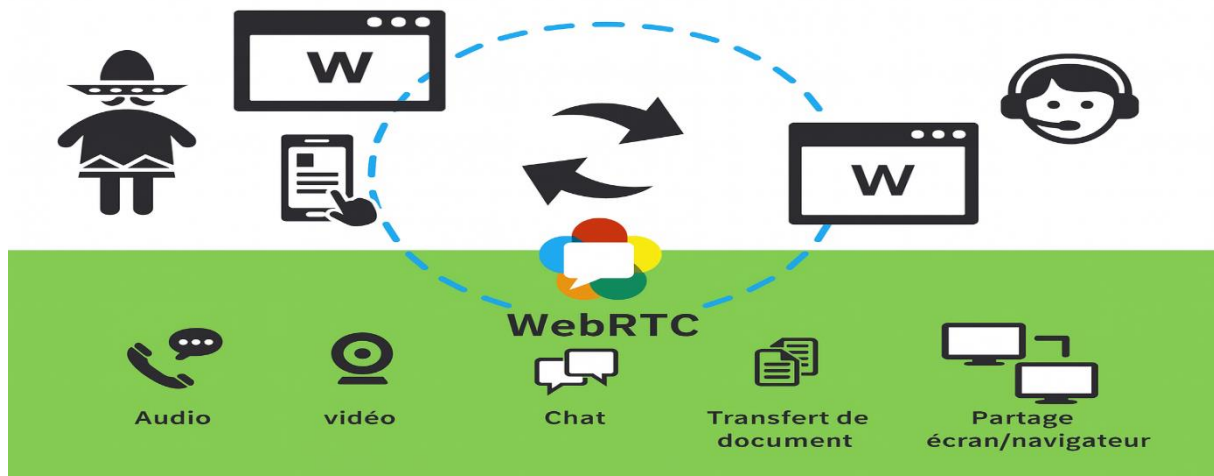


Figure 6 : Les fonctionnalités du WebRTC (Source : Google)

I.1.2.2.2 Les avantages de la technologie WebRTC

La technologie WebRTC (Web Real-Time Communication) révolutionne la communication en ligne en permettant les appels audios, vidéo et l'échange de données en temps réel, directement depuis un navigateur web, sans nécessiter de plugin ou logiciel externe. Grâce à son caractère open source, sa compatibilité avec les navigateurs modernes et sa sécurité intégrée, WebRTC s'impose comme une solution flexible, économique et performante pour les applications de téléphonie IP, de visioconférence et de collaboration à distance.

Voici quelques avantages majeurs de la technologie WebRTC :

- ✚ Communication en temps réel : WebRTC permet l'audio, la vidéo et l'échange de données en temps réel, directement entre les navigateurs, sans plugin.

- ✚ Gratuit et open source : WebRTC est libre et soutenu par Google, et largement adapté, ce qui réduit les coûts de licence.
- ✚ Multiplateforme : Fonctionne sur les navigateurs modernes (Chrome, Firefox, Edge, Safari) et sur mobiles via des SDK.
- ✚ Qualité audio/vidéo optimisée : WebRTC intègre des codecs (Opus, VP8/VP9, etc.) pour une bonne qualité, même avec une faible bande passante.
- ✚ Sécurité intégrée : Utilise DTLS et SRTP pour chiffrer les flux. Les connexions WebSocket sont aussi sécurisées (WSS).
- ✚ Facile à intégrer dans les applications web : Utilise des API JavaScript simples (getUserMedia, RTCPeerConnection, etc) pour intégrer la communication.

I.1.2.2.3 Les composantes de la technologie WebRTC

Le WebRTC se décline en trois autres APIs assurant chacune une fonction spécifique :

- ✚ L'API `MediaStream` : Permet d'accéder aux périphériques multimédia (caméra, micro) du navigateur, avec le consentement de l'utilisateur.
- ✚ L'API `RTCPeerConnection` : Gère la connexion audio/vidéo entre deux pairs. C'est le cœur de la communication en temps réel : il établit, maintient, et sécurise la connexion.
- ✚ L'API `RTCDataChannel` : Permet d'envoyer et de recevoir des données génériques à travers un canal de communication pair à pair.

I.1.2.2.4 Limites de la technologie WebRTC

Bien qu'elle soit devenue un standard incontournable de nos jours pour la communication en temps réel, la technologie WebRTC présente plusieurs limites structurelles et opérationnelles :

- ✚ **Modèle Mesh limité** : Nativement, WebRTC utilise une architecture *peer-to-peer* (P2P) où chaque participant se connecte directement à tous les autres. Cela fonctionne pour des appels en tête-à-tête, mais devient insoutenable au-delà de 4 à 5 participants car la consommation de bande passante et de CPU croît de manière exponentielle.
- ✚ **Complexité des infrastructures tierces** : Pour dépasser ces limites, les développeurs doivent déployer des serveurs intermédiaires coûteux et complexes comme des **SFU** (*Selective Forwarding Units*) ou des **MCU** (*Multipoint Control Units*).

- ✚ **Traversée des pare-feu et NAT** : Établir une connexion directe entre deux appareils derrière des pare-feu d'entreprise ou des routeurs (NAT) échoue souvent.
- ✚ **Dépendance aux serveurs STUN/TURN** : Pour contourner ces blocages, l'utilisation de serveurs STUN et TURN est obligatoire, ce qui rajoute de la latence et des coûts d'infrastructure importants.
- ✚ **Complexité de mise en œuvre** : Bien que le flux soit chiffré par défaut, l'absence de standard imposé pour la partie "signalisation" laisse aux développeurs la responsabilité de sécuriser l'échange initial, ce qui mène souvent à des failles applicatives

I.1.2.2.5 Types d'architecture de conférence WebRTC

Les types de conférence WebRTC font généralement référence à la différente architecture de serveurs (comme les SFU ou les MCU) et aux types de déploiements qui permettent les conférences vidéo. Les PDF pertinents décrivent souvent ces architectures, comme une étude comparative d'Open Source SFUs (Selective Forwarding Units) pour des conférences vidéo, ou des documents sur la mise en œuvre de solutions de conférence basées sur WebRTC.

Il existe trois principaux techniques réseaux permettant de mettre en œuvre la visioconférence : le mode Mesh sans serveur relai et deux modes faisant intervenir des serveurs relai, SFU et MCU.

✚ **L'architecture Mesh**

Le mode Mesh (maillage en français, encore appelé P2P ou peer to peer). Dans ce modèle, chaque participant établit une connexion directe avec tous les autres participants, sans serveur central. Cette approche est simple mais ne scale pas bien avec un grand nombre de participants.

Le mode Mesh met directement en relation les participants à la visioconférence : les flux ne sont pas relayés par un serveur central mais sont routés directement par le réseau. Il ne nécessite pas de serveur, ce qui économise des coûts d'hébergement.

Chaque participant envoie ses flux audio et vidéo à chacun des autres participants et mobilisé sa bande passante montante (upload) proportionnellement au nombre de participants. Ce mode est adapté jusqu'à quatre participants en général. En effet, la bande passante étant souvent asymétrique (le « A » de ADSL), celle-ci est limitée dans le sens montant donc vite saturer. Par exemple, une conférence à 4 participants nécessite pour chaque participant 3 Mbit/s dans le sens montant aussi bien que descendant (download).

En résumé, dans ce mode, pour une conférence à n participant envoie et reçoit $n-1$ flux de 1Mbit/s.

Le mode Mesh est utilisé en général pour les communications où il y a que deux à quatre participants.

L'architecture SFU

Architecture basée sur un serveur de retransmission SFU (Selective Forwarding Unit « Unité de relai sélective ») : L'utilisateur envoie son flux vidéo et audio une seule fois à un SFU. Le SFU retransmet ensuite les flux de chaque participant aux autres participants. C'est le modèle le plus courant pour les conférences avec de nombreux participants, car il optimise l'utilisation de la bande passante et des ressources de chaque participant.

Le serveur SFU a besoin pour sa part d'une bande passante sortante conséquente car il redistribue les flux à chaque participant : cela croît quasiment autant que le carré du nombre de participant. Ainsi avec 10 participants à 1 Mbits/s, le SFU sort 10×9 (il ne renvoie pas au participant son propre flux) soit 90 Mbits/s.

En résumé, dans ce mode, pour une conférence à n participants, chaque participant envoie 1 flux de 1 Mbits/s et reçoit $n-1$ flux de 1 Mbits/s. le serveur SFU reçoit n flux de 1 Mbits/s et envoie $n \times (n-1)$ flux de 1 Mbits/s.

C'est tout à fait acceptable pour un serveur dans un data center. Il n'est pas rare en effet de disposer de 1 Gbits/s en sortie sur un serveur hébergé.

L'architecture SFU

MCU (Multipoint control Unit, littéralement « Unité de contrôle Multipoint ») : Le MCU reçoit les flux de tous les participants, les mélange en un seul flux vidéo, et envoie ce flux combiné à chaque participant. Ce modèle est moins courant car il consomme beaucoup de ressources sur le serveur.

Le serveur MCU centralise les flux de chaque participant et en effectue un mixage avant de renvoyer un unique flux à chaque participant. L'opération de mixage est une opération intensive au niveau du serveur et monopolise une capacité de calcul importante liée au décodage et à l'encodage des flux.

En résumé, dans ce mode, pour une conférence à n participants, chaque participant envoie et reçoit un seul flux de 1 Mbits/s, le serveur reçoit et envoie n flux de 1 Mbits/s.

Voici une image explicative des trois architectures :

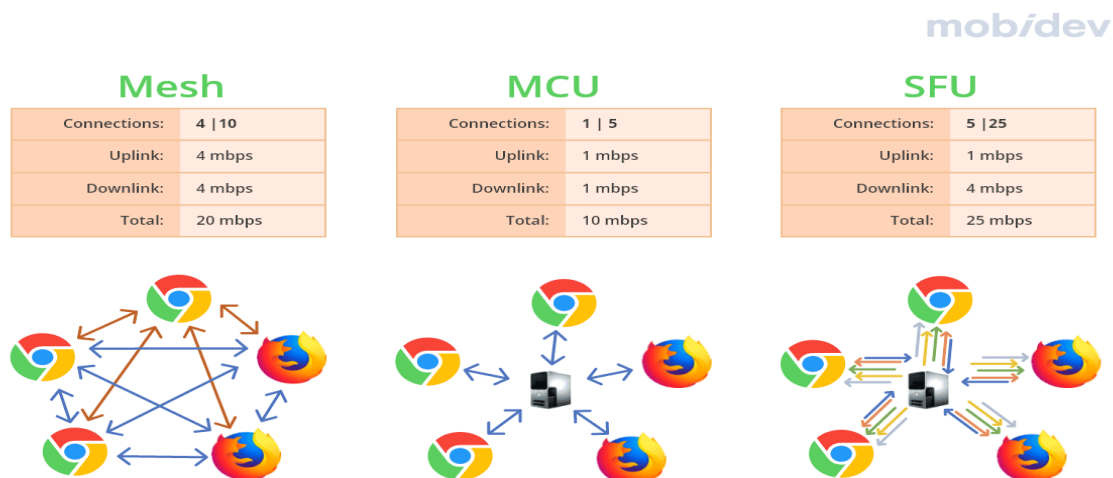


Figure 7 : Types d'architecture de conférence WebRTC (Source : Google)

I.1.2.3 Présentation d'Asterisk

Asterisk est une plateforme logicielle open source puissante qui transforme un ordinateur en un système téléphonique complet (IPBX), offrant des fonctionnalités avancées comme la messagerie vocale, les conférences, les files d'attente et la messagerie, et fonctionnant sur Linux/Unix pour des solutions de téléphonie sur IP (VoIP) flexibles et économiques. Créé par Mark Spencer, Asterisk supporte divers protocoles (SIP, IAX) et s'intègre aux réseaux existants, servant de base pour des applications complexes comme les centres d'appel



Figure 8 : Asterisk (Source :Google)

I.1.2.3.1 Architecture

L'Architecture modulaire et flexible d'Asterisk lui permet de fonctionner comme puissant IPBX, une passerelle VoIP ou un serveur d'application de communication personnalisées en assemblant les composants logiciels nécessaire.



Figure 9 : Architecture (Source : Google)

I.1.2.3.2 Fonctions principales dans la ToIP

Asterisk est la plateforme centrale qui gère toute l'infrastructure de téléphonie sur IP, en permettant de créer des solutions sur mesure, personnalisées et évolutives, contrairement aux solutions propriétaires. Il est configuré via des fichiers (comme extensions.conf, sip.conf) pour définir le plan de numérotation et les fonctionnalités des appels.

Voici quelques-unes les plus importantes :

-  **PABX Logiciel** : Il remplace les PABX matériels obsolètes, offrant plus de flexibilité et des coûts réduits
-  **Routage d'appels** : Connecte les postes internes (IP phones, softphones) et gère les appels vers l'extérieur.
-  **Services Avancés** : Intègre messagerie vocale, répondeur interactif (IVR), musique d'attente, files d'attente, conférences, transfert d'appels, etc...
-  **Passerelle** : Permet d'interconnecter le réseau IP avec les lignes téléphoniques traditionnelles (RTC/RNIS) et les opérateurs VoIP via des trunks SIP ou IAX.
-  **Intégration SI** : Se couple facilement avec les systèmes d'information de l'entreprise (annuaires, applications web).

I.1.2.3.3 Fonctionnement d'Asterisk

1. Un téléphone SIP appelle un numéro.
2. L'appel arrive sur un canal SIP d'Asterisk.
3. Le dialplan interprète le numéro et décide du routage (ex : renvoi vers un autre canal SIP ou une application).
4. L'appel est redirigé, potentiellement via d'autres canaux (IAX pour un autre site) ou une passerelle.
5. L'appel aboutit au destinataire (téléphone, boîte vocale, etc.).

Dans ce chapitre, nous avons parcouru certaines technologies de la visioconférence existantes. Ce qui nous permet de mieux comprendre la différence entre les solutions propriétaires et les solutions open source, la présentation des technologies utilisées.

CHAPITRE II : ANALYSE DES BESOINS ET SPÉCIFICATIONS

L'analyse des besoins constitue une étape fondamentale dans la conception de tout système de communication en temps réel. Dans le cadre de cette étude, il s'agit d'identifier de manière précise les attentes des utilisateurs ainsi que les contraintes techniques auxquelles doit répondre une architecture de visioconférence basée sur WebRTC et Asterisk. Une mauvaise identification des besoins peut entraîner une solution inadaptée, peu performante ou difficile à faire évoluer.

Ce chapitre vise donc à analyser en profondeur les besoins fonctionnels, les besoins non fonctionnels ainsi que les contraintes techniques du système proposé. Les résultats de cette analyse serviront de base pour la conception de l'architecture détaillée dans le chapitre suivant.

II.1 Identification et analyse des besoins fonctionnels

Les besoins fonctionnels représentent l'ensemble des services et fonctionnalités que le système doit offrir aux utilisateurs finaux. Ils sont définis à partir des usages réels observés dans les environnements professionnels, académiques et collaboratifs.

II.1.1 Besoin de communication audio en temps réel

La communication audio constitue la fonction essentielle de tout système de visioconférence. Elle permet aux utilisateurs d'échanger de manière fluide et instantanée, indépendamment de leur localisation géographique.

Le système doit permettre :

- L'établissement d'appels audio bidirectionnels en temps réel
- Une qualité sonore stable, claire et intelligible
- La gestion de l'activation et de la désactivation du microphone
- La réduction de l'écho, du bruit ambiant et des interférences
- La prise en charge des codecs audio adaptés aux réseaux IP modernes

Dans l'architecture proposée, WebRTC assure la capture et l'encodage du signal audio directement depuis le navigateur de l'utilisateur, tandis qu'Asterisk gère la signalisation, le routage des appels et la mise en relation des participants. Le codec Opus, largement utilisé dans WebRTC, permet une adaptation dynamique à la bande passante et offre une excellente qualité audio, même dans des conditions réseau dégradées.

II.1.2 Besoin de communication vidéo interactive

La communication vidéo est devenue un élément central dans les systèmes de collaboration à distance, car elle améliore l'interaction humaine et la compréhension visuelle entre les participants.

Le système doit répondre aux besoins suivants :

- Établir des appels vidéo en temps réel
- Offrir une synchronisation audio-vidéo correcte
- Adapter automatiquement la résolution et le débit vidéo en fonction de la qualité du réseau
- Permettre à l'utilisateur d'activer ou de désactiver sa caméra à tout moment
- Supporter les principaux codecs vidéo compatibles avec WebRTC (VP8, VP9, H.264)

WebRTC fournit nativement les mécanismes d'encodage, de décodage et d'adaptation du flux vidéo, ce qui réduit la complexité côté client. Asterisk, quant à lui, assure la gestion des sessions et la coordination des flux dans le cas des communications multipoints.

II.1.3 Besoin de conférence multi-utilisateurs

Dans un contexte professionnel ou académique, les communications ne se limitent pas à des échanges point à point. Le système doit permettre l'organisation de réunions virtuelles impliquant plusieurs participants simultanément.

Les fonctionnalités attendues incluent :

- La création de salles de conférence virtuelles
- La possibilité pour plusieurs utilisateurs de rejoindre une même session
- La gestion dynamique des entrées et sorties des participants
- La stabilité des communications, même avec un nombre élevé d'utilisateurs
- La possibilité de désigner un organisateur ou un modérateur

Asterisk joue ici un rôle central en tant que serveur de conférence, permettant la gestion des flux audio et vidéo et assurant la coordination entre les participants. Cette approche permet une meilleure maîtrise des ressources et une interopérabilité avec des terminaux SIP traditionnels.

II.1.4 Besoin de partage d'écran et de contenus

Le partage d'écran est une fonctionnalité essentielle pour les réunions, les formations à distance et les présentations professionnelles.

Le système doit permettre :

- Le partage de l'écran complet ou d'une application spécifique
- Une transmission fluide et en temps réel du contenu partagé
- Le maintien d'une bonne qualité visuelle
- L'arrêt du partage à tout moment par l'utilisateur

Cette fonctionnalité est directement fournie par WebRTC via les API du navigateur, ce qui permet une intégration simple et efficace sans logiciel supplémentaire.

II.1.5 Besoin de communication textuelle (chat)

La messagerie instantanée constitue un complément important aux communications audio et vidéo. Elle permet l'échange d'informations textuelles sans interrompre la communication principale.

Le système doit offrir :

- L'envoi et la réception de messages en temps réel
- La possibilité d'échanger des informations complémentaires
- Une synchronisation avec les sessions audio et vidéo

Le chat peut être implémenté à l'aide de WebRTC DataChannels ou via WebSocket, assurant ainsi une communication rapide et sécurisée.

II.1.6 Besoin de gestion des utilisateurs et des sessions

Un système de visioconférence fiable doit assurer une gestion efficace des utilisateurs et des sessions de communication.

Les besoins incluent :

- L'authentification sécurisée des utilisateurs
- La gestion des profils utilisateurs
- La création, la supervision et la terminaison des sessions
- La journalisation des appels et des connexions

Asterisk fournit des mécanismes robustes pour la gestion des comptes SIP et le suivi des sessions actives.

II.2 Analyse des besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité et de performance du système.

II.2.1 Besoins en matière de sécurité

La sécurité est un enjeu majeur pour les systèmes de visioconférence, notamment en raison de la sensibilité des données échangées.

Le système doit garantir :

- Le chiffrement des flux de signalisation
- Le chiffrement des flux audio et vidéo
- Une authentification forte des utilisateurs
- La protection contre les écoutes non autorisées et les attaques réseau
- L'utilisation de DTLS-SRTP et de SIP sécurisé permet de répondre à ces exigences.

II.2.2 Besoins de performance et de qualité de service

Pour offrir une bonne expérience utilisateur, le système doit :

Présenter une faible latence

S'adapter aux variations de la bande passante

Minimiser la perte de paquets et le jitter

Maintenir une synchronisation audio-vidéo correcte.

II.2.3 Besoins d'évolutivité et de disponibilité

Le système doit être conçu de manière à :

- Supporter un nombre croissant d'utilisateurs
- Permettre une évolution progressive de l'architecture
- Assurer une disponibilité continue du service

II.3 Contraintes techniques et environnementales

II.3.1 Contraintes réseau

Les communications en temps réel sont fortement dépendantes des conditions réseau. Le système doit fonctionner malgré :

- La présence de NAT et de pare-feu
- Les variations de bande passante
- L'hétérogénéité des réseaux d'accès
- L'intégration de STUN, TURN et ICE est donc indispensable.

1. Contraintes liées aux protocoles

Le système repose sur plusieurs protocoles standards :

- SIP pour la signalisation
- RTP/SRTP pour le transport des médias
- WebSocket pour la communication avec les clients WebRTC

II.3.2 Contraintes matérielles et logicielles

Le déploiement du système nécessite :

- Un serveur Linux pour Asterisk
- Des navigateurs compatibles WebRTC
- Des ressources matérielles suffisantes pour supporter la charge

Ce chapitre a permis d'analyser en profondeur les besoins fonctionnels, non fonctionnels et techniques du système de visioconférence proposé. Cette analyse constitue un élément clé dans la réussite du projet, car elle permet d'orienter les choix architecturaux et technologiques. Le chapitre suivant présentera la conception détaillée de l'architecture WebRTC–Asterisk, élaborée pour répondre efficacement à l'ensemble des besoins identifiés

CHAPITRE III : CONCEPTION DE L'ARCHITECTURE

Après avoir identifié et analysé les besoins fonctionnels, non fonctionnels et les contraintes techniques dans le chapitre précédent, il est maintenant essentiel de concevoir une architecture capable de répondre à ces exigences. La conception définit les composants principaux, leur organisation, les flux de communication et les modules complémentaires qui permettront au système de fonctionner de manière efficace, sécurisée et évolutive.

L'architecture proposée s'appuie sur WebRTC côté client pour gérer les médias en temps réel et sur Asterisk côté serveur pour la signalisation, la gestion des sessions et des conférences.

III.1 Schéma global de l'architecture

L'architecture du système peut être représentée par trois couches principales :

- a. Couches clients : navigateurs ou applications WebRTC des utilisateurs
- b. Serveur de signalisation et média : Asterisk
- c. Infrastructure réseau : NAT, pare-feu, serveurs STUN/TURN

III.1.1 Composants principaux

Clients WebRTC :

- ✚ Capturent et restituent l'audio, la vidéo et l'écran
- ✚ Gèrent la messagerie instantanée via DataChannel
- ✚ Implémentent la signalisation WebSocket vers Asterisk

Serveur Asterisk :

- ✚ Gère la signalisation SIP et WebSocket
- ✚ Coordonne les conférences audios et vidéo
- ✚ Gère les comptes utilisateurs et l'authentification
- ✚ Interface avec STUN/TURN pour NAT traversal

Serveurs STUN/TURN :

- ✚ Assurent la connectivité entre utilisateurs derrière NAT ou pare-feu
- ✚ STUN pour la découverte de l'adresse publique
- ✚ TURN pour le relay des médias si la connexion directe est impossible
- ✚ Base de données (optionnelle mais recommandée) :
- ✚ Stocke les comptes utilisateurs, historiques de sessions et configurations

III.1.2 Schéma conceptuel

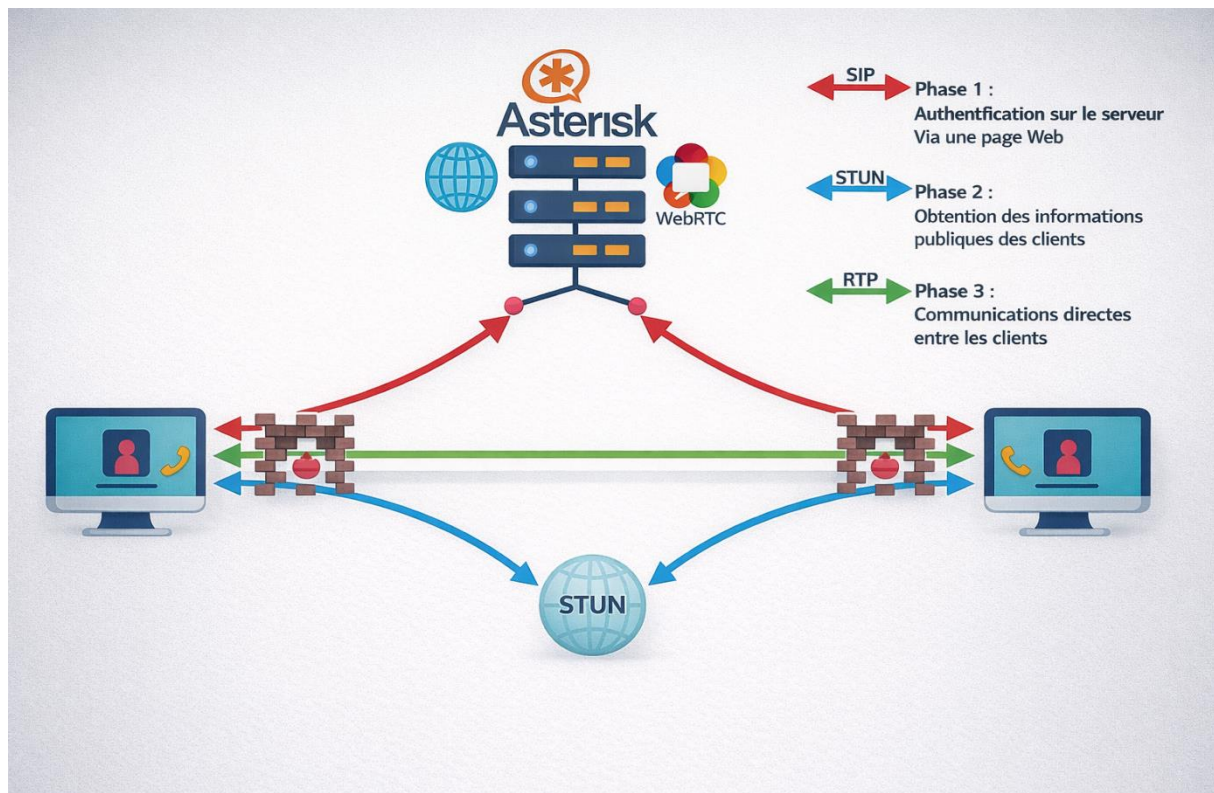


Figure 10 : Schéma conceptuel (Source : Google)

Les flèches représentent les flux de données :

- Flux média (audio, vidéo, partage d'écran) : WebRTC ↔ WebRTC via Asterisk
- Signalisation et authentification : WebSocket → Asterisk
- Gestion des sessions et utilisateurs : Asterisk ↔ Base de données

III.2 Flux de communication

En téléphonie sur IP plusieurs protocoles sont utilisés pour établir, gérer et transmettre les appels

La communication dans le système repose sur deux types principaux de flux :

III.2.1 Signalisation

Un protocole de signalisation est celui qui permet d'établir, maintenir et fermer une communication entre deux postes (téléphonique) terminaux. En ToIP, on trouvera plusieurs protocoles de signalisation. Les principaux sont :

-  SIP
-  H.323
-  MGCP
-  IAX

Protocole SIP

SIP sur Websocket (WSS) est utilisé pour échanger les informations d'appel entre les clients et Asterisk.

SIP (Session Initiation Protocol) est un protocole de signalisation utilisé dans le réseau IP pour établir, modifier et terminer la session de communications telles que les appels VoIP, la visioconférence, la messagerie instantanée et d'autres applications multimédias.

SIP est un protocole de signalisation appartenant à la couche transport du modèle OSI, son rôle est d'ouvrir, modifier et libérer les sessions. L'ouverture de ces sessions permet de réaliser de l'audio ou de vidéoconférence, de l'enseignement à distance, de la voix (téléphonie) et de la diffusion multimédia sur IP. Enfin SIP possède l'avantage de ne pas être attaché à un médium particulier et est censé être indépendant du protocole de transport des couches basses.

Les messages utilisés par SIP sont volontairement similaires à ceux utilisés par le http. Ils sont codés en ASCII et utilisent des codes proches de http. Différents messages sont utilisés par SIP, les plus importants sont les suivants

❖ REGISTER :

Le client envoie ce message à son registrar pour s'enregistrer, c'est-à-dire pour donner URI et son adresse IP au registrar.

❖ INVITE :

Ce message permet à un client de demander l'établissement d'une nouvelle session. Il peut être utilisé également en cours de communication pour modifier la session.

❖ ACK

Le message ACK confirme l'établissement d'une session SIP, suite à un message INVITE.

❖ CANCEL :

Ce message annule une demande de session précédemment effectuée avec un INVITE.

❖ **BYE :**

Ce message termine une session en cours. Contrairement au message CANCEL, la session SIP doit être active pour pouvoir envoyer un message BAY. Même si son comportement peut sembler similaire à CANCEL, une différence fondamentale existe : le message BAY représente un succès (la communication a eu lieu et désormais terminée) alors que CANCEL est, du point de vue de l'utilisateur, un échec d'appel n'a pas répondu à temps, l'appelant à donc raccroché, la session n'a donc pas abouti.

Les codes de réponse :

Des codes à trois chiffres, similaires à ceux de http, sont envoyés en réponse à un précédent message. Le premier chiffre détermine le type de réponse, les deux suivants donnent une indication plus précise.

1xx- réponse temporaire : la requête est cours de traitement.

2xx- Succès : l'action demander a été reçue, comprise et acceptée.

3xx- Redirection : une autre action auprès d'un autre équipement est nécessaire.

4xx- Erreur du client : la requête est mal formée.

5xx- Erreur du serveur : le serveur n'a pas réussi à répondre correctement à la requête.

6xx- Autre erreur : problème global.

Déroulement d'un appel téléphonique sous SIP

Pour initier une session sous SIP, la procédure est la suivante : les messages ne transitent pas par un proxy mais sont envoyés directement à l'User Agent concerné. L'appelant envoie un INVITE à l'appelé. Au moyen du protocole SDP, il indique dans sa requête quel media il souhaite échanger (audio/vidéo) et les codecs qu'il prend en charge.

L'appelé lui indique qu'il traite la requête par un code de réponse '100 trying'. Une fois la requête traitée, il envoie un code '180 ringing' pour indiquer que le téléphone appelé est entrain de sonner.

Les réponses 100 et 180 sont généralement envoyées à quelques millisecondes d'intervalle seulement. Pour comprendre la réelle utilité du message '100 trying', il faut considérer le cas où l'appel à besoin de plus de temps pour traiter la requête s'il doit demander une autorisation avant d'accepter l'appel.

Lorsque l'utilisateur appelé répond, une réponse définitive est envoyée à l'appelant dans un message '200 Ok'. Ce message contient des informations de session grâce à SDP.

Contrairement au SDP du message INVITE, celui-ci ne contient qu'un seul codec car l'appelé a choisi le plus approprié dans la liste proposée par l'appelant. L'appelant termine par un message ACK, indiquant que la session est établie et les deux parties sont maintenant en communication.

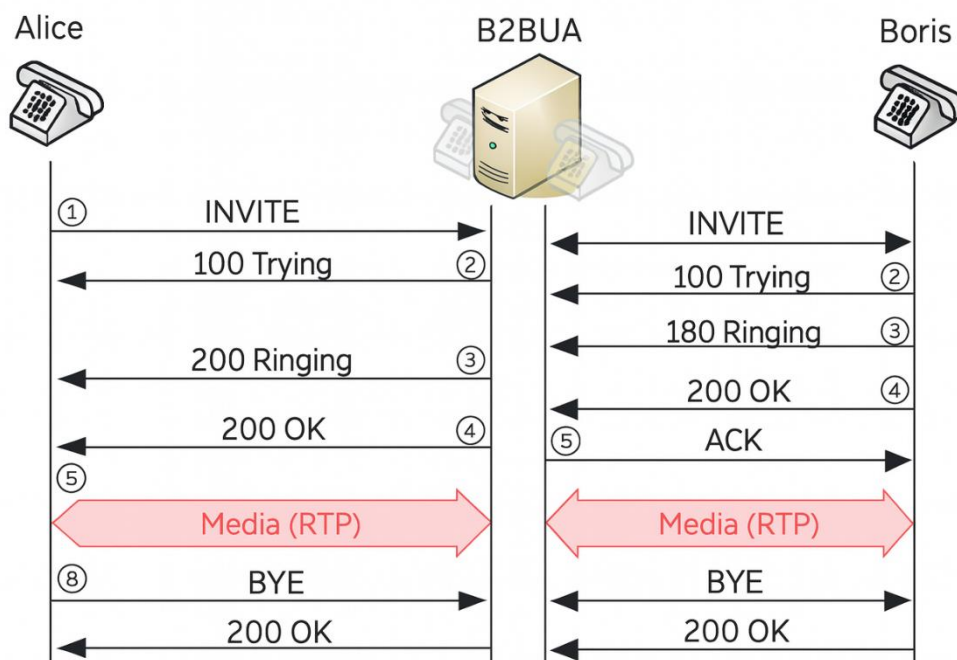


Figure 11 : Protocole SIP (Source : Google)

Protocole H.323

Le protocole H.323 est un protocole appartenant à la couche transport du modèle OSI et associant un ensemble de protocole de communication de voix, d'image et des données sur IP. Ce protocole est une adaptation de H.320 UTILIS2 SUR RNIS pour les réseaux IP.

H.323 est un protocole de communication englobe un ensemble de normes utilisées pour l'envoi de données audio et vidéo sur internet. Il existe depuis 1996 et a été initié par l'IUT. Concrètement, il est utilisé dans les programmes tels que Microsoft NetMeeting, ou encore dans des équipements tels que les routeurs Cisco.

L'infrastructure H.323 repose sur des éléments réseaux suivants :

- ✓ Les portiers (gk : Gatekeeper)

- ✓ Les passerelles (gw : Gateway)
- ✓ Les terminaux : Dans un contexte de téléphonie sur IP, deux types de terminaux H.323 sont aujourd'hui disponibles :

*Un poste téléphonique IP raccordés directement au réseau Ethernet de l'entreprise.

*Un PC multimédia sur lequel est installée une application compatible h.323

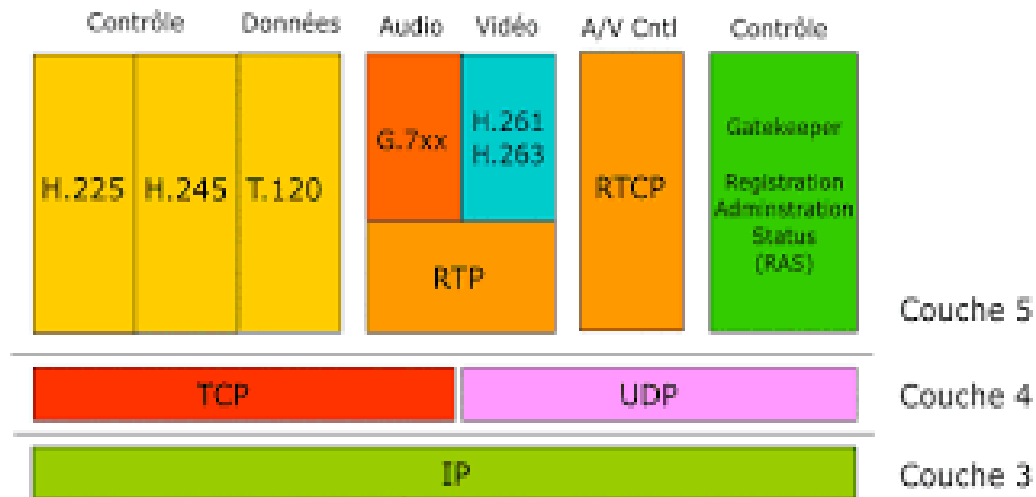


Figure 212 : Protocole H.323 (Source : Google)

Protocole IAX

IAX (Inter-Asterisk eXchange) : c'est un protocole développé pour le logiciel Asterisk, souvent utilisé pour les communications VoIP. Sa principale différence avec SIP vient de sa capacité à contrôler et réguler la transmission de flux multimédia avec un débit faible pour la voix. Il présente aussi l'avantage de s'intégrer dans le réseau NAT, et en effet IAX utilise qu'un seul port UDP : le 4569 pour la signalisation et les données. Le nom IAX est souvent utilisé pour parler de la version 2 du protocole, en effet la première version n'est pratiquement pas utilisée.

Protocole MGCP

Le MGCP qui signifie (Media Gateway Control Protocole) est un protocole qui permet à un serveur de contrôler les passerelles multimédias souvent utilisées pour interconnecter les réseaux téléphoniques traditionnels et IP.

Le protocole MGCP n'est pas un concurrent de H.323 ou SIP. Au contraire, ce protocole est complémentaire dans le service qu'il offre. Il permet entre et autre de piloter les terminaux non

intelligents. Pour cela, MGCP utilise un protocole à stimulus. Ce type de signalisation utilise des instructions de bas niveau contrairement aux instructions du RNIS, SIP et de H.323. Dans un protocole stimulus, le terminal est contrôlé par le protocole. MGCP est donc un protocole client-serveur car la machine contrôle les terminaux H.323 et SIP sont peer to peer car ils traitent d'égaux à égaux.

	H.323	SIP	MGCP
Inspiration	Téléphonie	http	
Complexité	Elevée	Faible	Elevée
Adaptation/Modularité protocolaires	Faible	Elevée	Modérée
Implémentation de nouveaux Service	Non	Oui	Non
Adapté à l'internet	Non	Oui	Non
Protocole de Transport	TCP	TCP ou UDP	TCP ou UDP
Coût	Elevé	Faible	Modéré

Tableau 1 : Comparaison entre les différents protocoles de signalisation
(Source : Google)

III.2.2 Transport des médias

En plus de différents types de protocoles de signalisation, il existe également des différents protocoles de transports. Un protocole de transport permet de transporter des données dans un réseau. Il existe plusieurs protocoles de transport mais voici quelques-uns les plus les plus importantes

RTP/RTCP et SRTP est utilisé pour l'audio et la vidéo, WebRTC gère la capture et le rendu des flux média, Asterisk peut agir comme pont pour les conférences multipoints et STUN/TURN est utilisé pour traverser les NAT et assurer la connectivité directe.

Le protocole RTP

Le RTP est un protocole qui se situe au niveau de l'application et qui utilise le protocole sous-jacent de transport TCP ou UDP. Mais l'utilisation de RTP se fait généralement au-dessus de l'UDP ce qui permet d'atteindre plus facilement le temps réel.

Les applications en temps réels comme la parole numérique ou la visioconférence constituent un véritable problème pour l'internet.

RTP (Real-time Transmission Protocol) : Ce protocole est spécifiquement conçu pour le transport de données en temps réel, comme l'audio et la vidéo. Son rôle est d'encapsuler les paquets de médias (la voix numérisée) et leur ajouter des informations cruciales telles qu'un

- ❖ Numéro de séquence (pour réordonner les paquets à l'arrivée),
- ❖ Horodatage (pour synchroniser la lecture)
- ❖ Identifiant de source.

Le protocole RTCP

RTCP (Real-time Transport Control protocol) est un protocole fondé sur la transmission périodique de paquet de contrôle à tous les participants d'une session.

Le RTCP c'est un protocole qui ne transporte pas les données vocales elles-mêmes, mais des informations de contrôle sur la session RTP. Voici les trois fonctions principales de RTCP :

- ❖ Fournir des informations sur la qualité de session
- ❖ Contrôle des débits auquel les participants à une session RTP transmettent leurs paquets RTCP
- ❖ Transmettre des informations de contrôle de session.

Le protocole TCP

Dans la téléphonie sur IP, certains protocoles de transports ne sont pas adaptés. Il s'agit particulièrement du protocole TCP (Transport Contrôle Protocol). Les raisons qui expliquent l'exclusion de ce protocole sont dues en particulier aux contraintes temporelles. TCP est un protocole de transport qui fonctionne en mode connecté (établissement de connexion et accusé de réception). Le TCP est utilisé dans la VoIP pour la signalisation des protocoles comme SIP et H.323 qui sont responsable de l'établissement, la modification et de la fin d'appels (sonnerie, mise en attente, ect...). TCP offre un service de transmission de données fiable avec détection et correction d'erreur de bout en bout.

Le protocole UDP

UDP (User Datagram Protocol) contrairement à TCP offre un service de transmission non fiable c'est-à-dire en mode non connecté. Bien que UDP présente quelque bonne caractéristiques, ce protocole ne séquence pas les données, donc ne garantit pas la remise conforme des données. Le protocole UDP satisfait au contrainte temporelle de la ToIP, mais n'ai pas adapté à cause du manque de contrôle (aucune garantie que les données transmis seront arrivées à destination dans leur intégralité). Malgré ces limites, mis c'est crucial pour la

téléphonie sur IP car sa faible surcharge (overhead) et son absence de mécanisme de retransmission le rendent extrêmement rapide.

III.3 Intégration des modules

Pour rendre l'architecture complète et robuste, plusieurs modules sont intégrés :

III.3.1 Module d'authentification

Les modules d'authentification sont des composantes qui permettent de vérifier l'identité d'un utilisateur avant de lui accorder l'accès à un système ou service.

Dans notre contexte, les modules d'authentification permettent de sécuriser des clients (comme les softphones ou clients webRTC). Voici les principaux éléments utilisés :

1. Authentification SIP (PJSIP) dans pjsip.conf
2. Association avec l'AOR (Address of Record)
3. Endpoint (Point de communication)
4. Authentification TLS/WebRTC

III.3.2 Gestion des sessions

La gestion des sessions fait référence au suivi des connexions actives entre les utilisateurs voici les points clés :

- Définition d'une session

Une session=une communication active entre deux terminaux (ex : 100 et 200)

- Dans Asterisk (via PJSIP/WebRTC) chaque appel crée une session SIP.

Le serveur gère :

- Crée, supervise et termine les appels
- Gérer les conférences multi-utilisateurs
- Journalise les activités pour audit et statistiques

III.3.3 NAT traversal (STUN/TURN)

ICE (Interactive Connectivity Establishment)

La signalisation nécessite l'utilisation initiale d'un serveur intermédiaire pour l'échange de métadonnées, mais à la fin, webrtc tente d'établir une connexion P2P directe entre les utilisateurs. Ce processus est réalisé à travers le cadre ICE.

ICE est un cadre utilisé pour établir une connexion entre pairs sur internet. Bien que webrtc essaie d'utiliser des connexions P2P directes, en réalité, la présence répandue de NAT (Network Address Translation) rend difficile la négociation de la manière dont deux pairs communiqueront.

En raison de la prévalence toujours répandue des adresses ipv4 avec leurs représentations limitées à 32 bits, la plupart des appareils compatibles au réseau n'ont l'adresse ipv4 publique unique avec laquelle elle serait directement visible sur l'internet. NAT fonctionne en traduisant dynamiquement l'adresse privée en adresse publiques lorsqu'une demande sortante les traverse.

De même les demandes entrantes vers une adresse IP publique sont reconverties en une adresse IP privée pour garantir un routage correct sur le réseau internet. Par conséquent, le partage d'une adresse IP privée n'est souvent pas assez d'information pour établir une connexion avec un homologue. ICE tente de surmonter les difficultés posées par la communication via NAT pour trouver le meilleur chemin pour connecter les pairs.

En essayant toutes les possibilités en parallèle. ICE est en mesure de choisir l'option la plus efficace qui fonctionne. ICE essaie d'abord d'établir une connexion en utilisant l'adresse hôte obtenue à partir du système d'exploitation et de la carte réseau d'un périphérique, si cela échoue (ce qui sera inévitablement le cas pour les périphériques derrière les NAT), ICE obtient alors une adresse externe en utilisant un serveur STN. Si cela échoue également, le trafic revient au routage via un serveur relais TURN.

Les voies de communication candidates sont rendues dans un format texte et la liste est classé par priorité. Les options prennent la forme de l'une des options suivantes :

- ✚ Communication P2P directe
- ✚ Utilisation de STUN, avec un mappage de port pour la traversée NAT (cette route se résout finalement en communication P2P directe)
- ✚ Utilisation de TURN comme intermédiaire (cette configuration utilise la communication relayée plutôt que P2P)

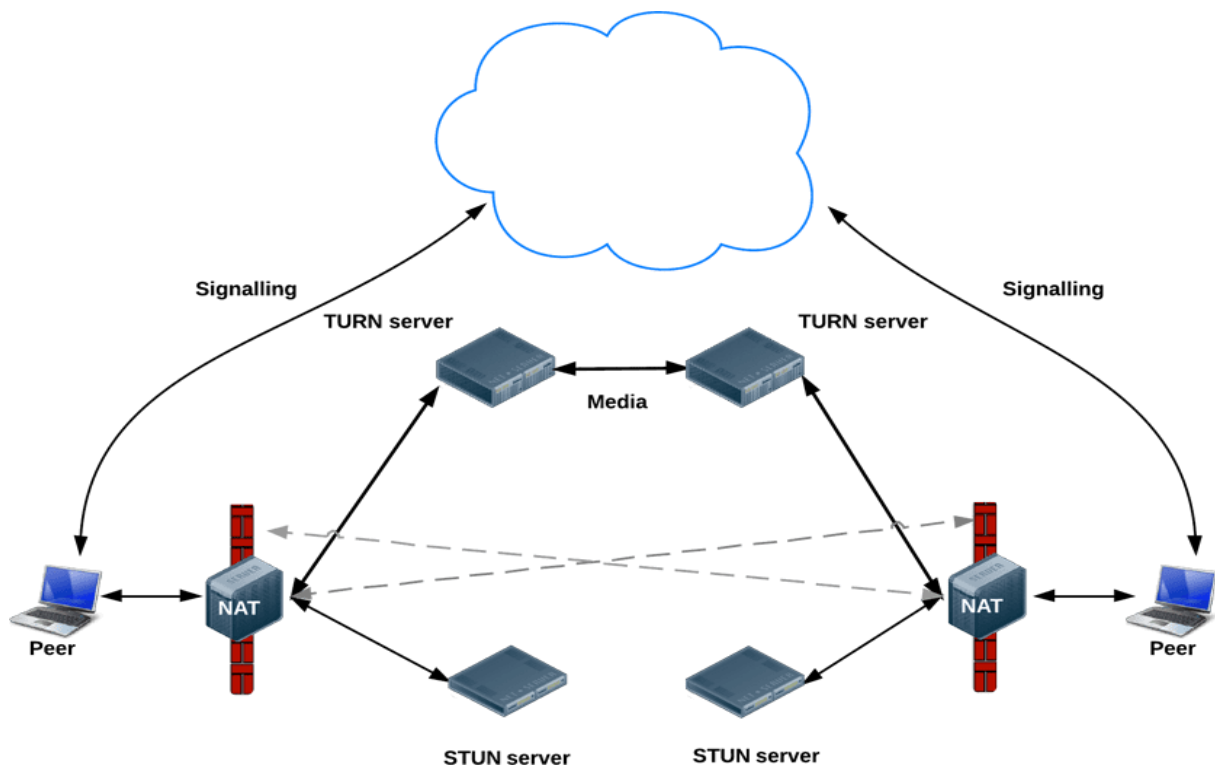


Figure 13 : NAT traversal (STUN/TURN) (Source : Google)

STUN (Session Traversal Utilities for NAT)

Pour effectuer une communication P2P, les deux parties doivent nécessairement connaître au moins l'adresse IP de leur homologue et le port UDP attribué. En conséquence, un certain échange d'informations est nécessaire avant que la communication webrtc puisse être établie.

Un serveur STUN (Simple Traversal of User Datagram protocol [UDP] Through Network Address Translations [NATs] ou traversée simple de UDP à travers les NATs) est utilisé par chaque homologue pour déterminer son adresse IP publique et est référencé par le cadre ICE lors de l'établissement de la connexion. Les serveurs STUN sont généralement accessibles au public et peuvent être utilisés librement par les applications webrtc.

TURN (Traversal Using Relays around NAT)

Dans l'éventualité où l'établissement d'une communication P2P échoue, une option de secours peut être fournie via un serveur TURN. En relayant le trafic entre homologues, la communication webrtc peut être assurée, mais peut souffrir des dégradations de la qualité de médias et de la latence. Les serveurs TURN peuvent garantir un succès élevé dans la

Configuration des appels, quel que soit l'environnement de l'utilisateur final. Comme les données sont envoyées via un serveur intermédiaire, la bande du serveur est également consommée. Si les de nombreux appels sont acheminés simultanément via le serveur, la bande passante était également devenue de taille considérable.

Le serveur lui-même n'est généralement pas librement accessible et doit être spécifiquement fourni (ou loué) par le fournisseur d'application.

III.3.4 Module de gestion des flux et QoS

Le module de gestion des flux et de la Qualité de service (QoS) joue un rôle essentiel dans la garantie d'une communication fluide et stable. Il est responsable de l'optimisation des flux audio en fonction des conditions réseau, telles que la bande passante disponible, la latence et la perte de paquets.

Un codec est un dispositif ou un programme informatique capable de compresser et de décompresser des données numériques, en particulier des signaux audios et vidéo. Le mot codec est une abréviation de codeur-décodeur qui permet de réduire la taille de fichiers volumineux pour en faciliter le stockage et la transmission sur des réseaux comme internet.

Codecs audios

Le code G.723

Le G.723 est un codec vocal standard de l'IUT-T utilisant des extensions du G.721 fournissant une quantité vocale couvrant de 300Hz à 3400Hz en utilisant la modulation adaptative différentielle à impulsion codées à 24 et 40 kbit/s pour les équipements de circuits numériques applications. La norme G.723 est obsolète et a été remplacée par G.726.

Le code G.711

Le G.711 est une norme recommandée par UIT-T pour le companding audio, intitulée modulation de code d'impulsion (PMD) des fréquences vocales, c'est une norme requise dans de nombreuses technologies, comme dans les normes H.320 et H.323. il a été initialement conçu pour une utilisation en téléphonie et a été publié G.711. Il peut être également être utilisé pour la communication de télécopie sur des réseaux IP (comme défini dans la spécification T.38). G.711 est une bande étroite codec audio qui fournit un son de qualité à 64 kbit/s. transmet les signaux audios dans la plage de 300 à 3400 Hz.

Le codec G.729

Le G.729 est un algorithme de compression de données audio basé sur un vocodeur à bande étroite libre de droits utilisant une longueur de trame de 10 millisecondes. Il est officiellement décrit comme le codage de la parole à 8 kbit/s à l'aide du codage de la parole prédiction linéaire excitée par le code (CS-ACELP). L'extension à large bande du G.729 est appelée G.729 est principalement utilisé dans les applications de voix sur protocole internet (VoIP) lorsque la bande passante doit être conservée. La norme G.729 fonctionne à un débit binaire de 8 kbit/s.

Le code Opus

Le code d'Opus est un discours et un codec audio interactif c'est-à-dire, conçu pour manipuler un large éventail d'applications sonores interactives telles que le VoIP, vidéoconférence, conversation de dans-jeu, et pour vivre représentation distribuée de musique. Les échelles de code d'Opus de 6 kbit/s de parole mono à bande étroite à 510 kbit/s de full bandent de musique de stéréo. Il peut sans faille commuter entre tous ses divers modes de fonctionnement, lui donnant beaucoup de flexibilité de s'adapter au contenu et aux états variables de réseau sans renégocier la session en cours.

Codecs	Débit	MOS
G.711	64 kbps	4,1
G.729	8 kbps	3,92
G.726	32 kbps	3,85
G.722	64 kbps	4,13
ILBC	15,2 kbps	4,1
Opus	6 kbps	4,8

Tableau 2 : Caractéristique de quelques codecs audios (*Source : Google*)

Codecs vidéo

Le codec H.261

Le codec H.261 est une recommandation de compression vidéo de l'UIT-T c'est une évolution de la norme H.120. Elle fut maintenant popularisée par la console de jeu playStation de Sony, qui l'intégra entièrement dans son moteur de décompression de données. Les travaux sur la norme H.261 ont ultérieurement servi de base à la norme MPEG. Cette norme devenue

obsolète, sa succession fut assurée par la norme H. 263 de meilleure qualité pour la VoIP ; pour le pur codage vidéo, on préféra la norme H.264. Et sa bande passante est de 64 kbit/s.

Codec H.263

Le codec H.263 est une recommandation concernant la norme de codage vidéo développé par l'UIT-T Q.6/SG16. A l'origine, H.263 a été développé pour la transmission de la vidéo sur des lignes de très bas débit, pour les applications de visiophonie via le réseau téléphonique commuté de type H.324. Elle a ensuite été intégrée dans les protocoles de visioconférence sur IP du type H.323 mais aussi SIP. Le même codec a été repris par la norme H.324m qui permet de réaliser des appels Visio en mode circuit sur les réseaux mobiles de troisième génération. Au début d'une communication vidéo entre deux terminaux équipés de ce codec, ils échangent

Leurs caractéristiques grâce au protocole H.245 et choisissent les modes de H.263 qu'ils utiliseront lors de la communication.

Codecs H.264

Le codec H.264 convertit les signaux audio et vidéo HDMI (HD) en flux IP pouvant être transmis sur un réseau IP. Un codeur convertit les signaux HDMI format non compressé. La polyvalence du H.264 réside dans la diffusion simultanée de la vidéo à partir d'un convertisseur vers plusieurs décodeurs. Vous pouvez ainsi diffuser en direct un ensemble des signaux vidéo vers un écran, et le système de signalisation numérique en même temps.

Codec VP8

Le codec VP8 est un code phare de Google qui en assure la promotion. Son taux de compression est comparable à H.264, il est open source et est devenu le protocole de référence pour plusieurs éditeurs du secteur. En revanche, il n'est pas encore imposé comme protocole qui devrait succéder à H.323/MPEG4. VP8 peut être multiplexé dans le format de conteneur WebM basé sur Matroska avec Vorbis et Opus audio. Le format d'image WebP est basé sur le codage intrame de VP8. Le successeur direct de VP8, VP9 est le nouveau format vidéo internet libre de droits. AV1 de l'alliance for Open Media (AOMedia) sont basés sur VP8.

III.4 Architecture détaillée côté serveur et client

Côté serveur (Asterisk) :

- SIP/WebSocket pour signalisation

- Gestion de ponts de conférence audio et vidéo
- Interface avec STUN/TURN et base de données

Côté client (webRTC) :

- Capture audio/vidéo et partage d'écran
- Affichage des flux entrants et gestion de la messagerie instantanée
- Gestion des événements de session et UI

L'architecture proposée combine WebRTC côté client et Asterisk côté serveur pour fournir un système de visioconférence sécurisé, évolutif et compatible avec différents environnements réseau. Les modules d'authentification, gestion des sessions et de traversée de NAT garantissent la fiabilité et la robustesse de la solution.

Cette conception servira de base pour le déploiement et la mise en œuvre, présenté dans le chapitre suivant.

CHAPITRE IV : DÉPLOIEMENT ET MISE EN ŒUVRE

Après la conception détaillée de l'architecture, il est essentiel de passer à la phase de déploiement et de mise en œuvre. Cette étape consiste à installer et configurer les composants logiciels et matériels, à déployer les clients WebRTC et à préparer l'environnement pour les tests. La réussite de cette phase garantit que le système est fonctionnel, sécurisé et performant.

Le déploiement comprend trois volets principaux : l'environnement de développement, l'installation et configuration du serveur Asterisk, et le déploiement des clients WebRTC.

IV.1 Environnement de développement

Avant d'installer Asterisk et WebRTC, il est nécessaire de préparer un environnement de développement adapté.

IV.1.1 Choix des outils et plateformes

- Système d'exploitation serveur : Linux (Ubuntu 22.04)
- Navigateurs Web : Chrome, Firefox ou Edge, compatibles WebRTC
- Serveurs STUN/TURN : Coturn pour le relais de médias derrière NAT

IV.1.2 Préparation du serveur

Mise à jour du système :

sudo apt update && sudo apt upgrade

Installation de dépendances :

Git, build-essential, libssl-dev, libncurses5-dev, libjansson-dev, uuid-dev

IV.2 Installation et configuration d'Asterisk

1. Téléchargement et compilation

Télécharger la dernière version stable :

wget http://downloads.asterisk.org/pub/telephony/asterisk/asterisk-20-current.tar.gz

tar xvfz asterisk-20-current.tar.gz

cd asterisk-20*/

./configure

make

sudo make install

sudo make samples

sudo make config

IV.2.1 Modules nécessaires pour WebRTC

Pour fonctionner correctement avec WebRTC, Asterisk doit être configuré pour accepter les connexions WebSocket sécurisées, gérer les flux chiffrés, et intégrer les protocoles nécessaires à la communication en temps réel à travers internet.

Nous allons activer la prise en charge de WebRTC dans la version 20.15.2 d'Asterisk après une bonne installation il faut au moins l'activation de quelques modules en plus des modules standards à savoir :

-  res_crypto
-  res_http_websocket : permet la communication WebSocket avec les clients WebRTC
-  res_pjsip_transport_websocket
-  codec_opus(facultatif mais fortement recommandé pour un son de haute qualité)

IV.2.2 Configuration des comptes utilisateurs

La configuration des utilisateurs s'effectue dans le fichier **pjsip.conf**. Ce fichier permet de déclarer des utilisateurs et les protocoles de transport utilisés pour les appels via pjsip. C'est un framework écrit en C et python permettant de développer des applications VoIP basées sur le protocole SIP.

```
[transport-wss]
```

```
type=transport
```

```
protocol=wss
```

```
bind=0.0.0.0:8089
```

```
method=tlsv1_2
```

```
[user1]
```

```
type=aor
```

max_contacts=5

remove_existing=yes

[user1]

type=auth

auth_type=userpass

username=user1

password=passdw1

[user1]

type=endpoint

aors=user1

auth=user1

dtls_auto_generate_cert=yes

webrtc=yes

context=default

disallow=all

allow=opus , ulaw , vp8

IV.2.3 Configuration du serveur HTTP pour WebRTC

Le fichier de configuration du serveur HTTP intégré à Asterisk est **http.conf**. Ce fichier contient les paramètres essentiels au bon fonctionnement du serveur Web et diverses options. Les principaux paramètres sont les suivants :

[general]

servername=Asterisk

enabled=yes

bindaddr=0.0.0.0

bindport=8088

tlsenable=yes

tlsbindaddr=0.0.0.0:8089

tlscertfile=/etc/asterisk/keys/asterisk.crt

tlsprivatekey=/etc/asterisk/keys/asterisk.key

- **Servername**= nom du serveur, il n'a pas d'influence sur le fonctionnement
- **enabled**= statut du serveur, yes pour l'activation, no pour désactiver
- **bindaddr**= réseau autorisé à se connecter au serveur
- **bindport**= port d'écoute du serveur, à utiliser pour se connecter
- **tlsenable**= activation du chiffrement des échanges via TLS
- **tlsbindaddr**= réseau et port d'écoute des connexions via TLS
- **tlscertfile**= adresse du certificat du site Web
- **tlsprivatekey**= adresse de la clé privée du serveur, utilisée pour le TLS

2. Configuration gestion du trafic

Le fichier **rtp.conf** nous permet de configurer la plage des ports utilisables par le protocole RTP ainsi que l'adresse et le port du serveur STUN qui gèrera les appels.

[general]

rtpstart=10000

rtpend=20000

icesupport=yes

stunaddr=stun.l.google.com :19302

Les paramètres ont chacun les rôles suivants :

- **Rtpstart** : Premier port de la plage utilisable pour les trames RTP.
- **Rtpend** : dernier port de la plage utilisable pour les trames RTP.

- **Icesupport** : activer la prise en charge de framework ICE
- **Stunaddr** : permet d'indiquer le serveur STUN à l'utiliser. Nous avons choisi d'utiliser un serveur en accès libre de Google (**stun.l.google.com :19302**).

ICE est utilisé par le WebRTC pour connecter deux entités, indépendamment de la topologie du réseau (utilisation de la translation d'adresse).

IV.3 Déploiement des clients WebRTC

Plutôt que de développer un client WebRTC maison, nous avons choisi d'utiliser SIPML5, un client open source compatible WebRTC

SIPML5

SIPml5 était un client SIP et un projet de communication en temps réel (RTC) open source écrit en JavaScript pour les navigateurs web, basé sur la technologie WebRTC pour les médias audio/vidéo.

Développé par Doubango Telecom, il était conçu pour se connecter à n'importe quel réseau SIP ou IMS et permettait de passer des appels audio/vidéo et d'envoyer des messages instantanés directement depuis un navigateur, sans extensions ni plugins.

Les données multimédia dépendent de WebRTC (Web Real Time Communication) qui est pris en charge par certains navigateurs. SIPML5 peut bien fonctionner sur tous les navigateurs prenant en charge WebRTC comme Google Chrome ou Firefox Nightly. En utilisant SIPML5 en conjonction avec WebRTC2Sip, nous pouvons passer des appels du navigateur vers les réseaux de télécommunications PSTN. Les interfaces de programmation SIPML5 sont conçues pour être les plus faciles à utiliser par les développeurs. L'utilisation et l'intégration de SIPML5 dans l'application peuvent se faire en quelques lignes de code.

L'interface est simple utilisation. Elle est composée de deux éléments : un formulaire d'enregistrement et une fenêtre d'appel.

Voici quelques fonctionnalités prise en charge :

Fonctionne sur Chrome, Firefox, Safari, Opera et Browser

- Authentification de l'utilisateur
- Établissement d'appel audio/vidéo
- Partage d'écran

- Chat instantané
- Gestion des conférences multipoints
- Transfert d'appel explicite
- Appel en attente/Reprise

IV.3.1 Tests de connectivité

- Vérifier la connexion aux serveurs STUN/TURN
- Tester l'établissement d'appels et la réception de flux
- Vérifier le bon chiffrement des flux (SRTP)

La phase de déploiement et mise en œuvre assure que le système WebRTC–Asterisk est opérationnel et conforme aux besoins identifiés. L'installation d'Asterisk, la configuration des modules WebRTC et le déploiement des clients garantissent une architecture prête pour les tests fonctionnels et de performance, qui seront détaillés dans le chapitre suivant.

CHAPITRE V : TESTS ET VALIDATION

Après la conception et le déploiement du système WebRTC–Asterisk, il est crucial d'évaluer son fonctionnement réel. Les tests permettent de vérifier si l'architecture répond aux besoins fonctionnels, non fonctionnels et techniques identifiées dans les chapitres précédents.

Ce chapitre décrit les différentes catégories de tests : fonctionnels, de performance et de sécurité, ainsi que les méthodes et critères utilisés pour valider le système.

V.1 Tests fonctionnels

Les tests fonctionnels permettent de vérifier que le système fournit bien toutes les fonctionnalités prévues.

Maintenant que notre serveur web a été bien configuré HTTPS en on peut tester avec un navigateur compatible WebRTC comme Chrome en mettant dans la barre de URL

<https://192.168.1.8>

On clique sur avancer puis continuer ver un site dangereux et on obtient la page d'accueil de sipml5

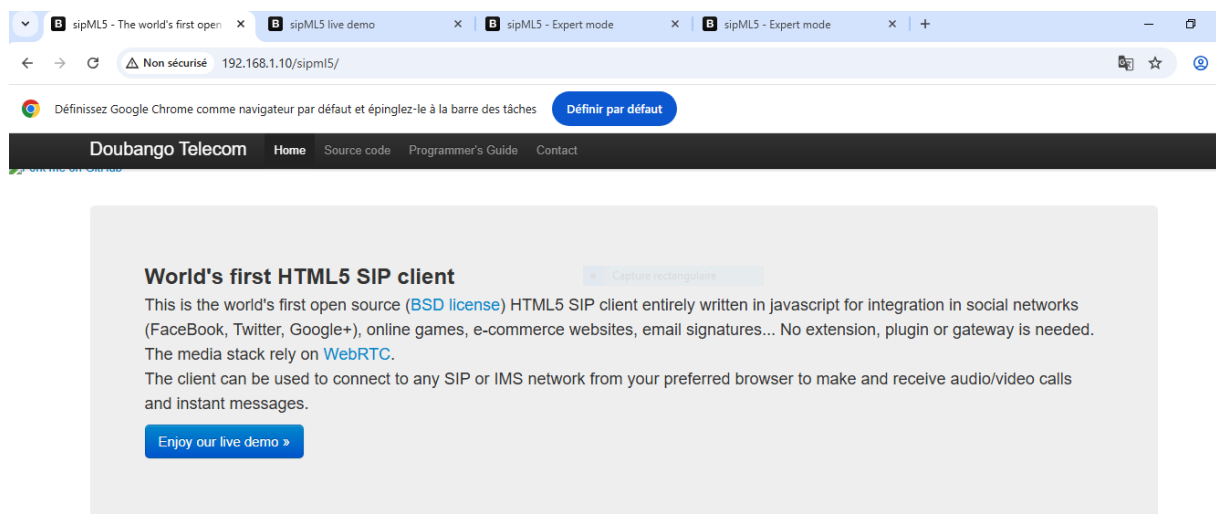


Figure 14 : Page d'accueil de sipml5 (Source : Nous-même)

On clique sur profiter de notre « demon en direct » puis aller en mode expert pour renseigner l'URL du serveur WebSocket avec wss et l'URL du transport

Saved

Expert settings

Disable Video:	☐
Enable RTCWeb Breaker ^[1] :	☐
WebSocket Server URL ^[2] :	wss://192.168.1.8:8089/ws
SIP outbound Proxy URL ^[3] :	e.g. udp://sipml5.org:5060
ICE Servers ^[4] :	<input "="" type="text" value="e.g. [{ url: 'stun:stun.l.google.com:19302'}, { url:'turn:user@"/>
Max bandwidth (kbps) ^[5] :	{ audio:64, video:512 }
Video size ^[6] :	{ minWidth: 640, minHeight:480, maxWidth: 640, maxHeigl
Disable 3GPP Early IMS ^[7] :	☒
Disable debug messages ^[8] :	☒
Cache the media stream ^[9] :	☒
Disable Call button options ^[10] :	☐

Figure 15 : Mode expert (*Source : Nous-même*)

Ensuite en revient pour se connecter avec un des utilisateurs 100 avec ses paramètres

The image shows a web-based registration form titled "Registration" with a "Connected" status at the top. The form contains five input fields: "Display Name" (containing "User"), "Private Identity*" (containing "user1"), "Public Identity*" (containing "sip:user1@192.168.1.8"), "Password" (containing "*****"), and "Realm*" (containing "192.168.1.8"). Below the fields are two buttons: a green "Login" button and a red "Logout" button. A note indicates that fields marked with an asterisk are mandatory. At the bottom of the form are two more buttons: "Need SIP account?" and "Expert mode?".

© Doubango Telecom 2012-2018
Inspiring the future

Figure 16 : Formulaire d'enregistrement (*Source : Nous-même*)

Pour passer un appel, il faut se placer dans la section prévue à cet effet la page connexion : **call control**. Comme pour téléphoner, l'utilisateur doit renseigner le numero de téléphone de la personne qu'il veut joindre.

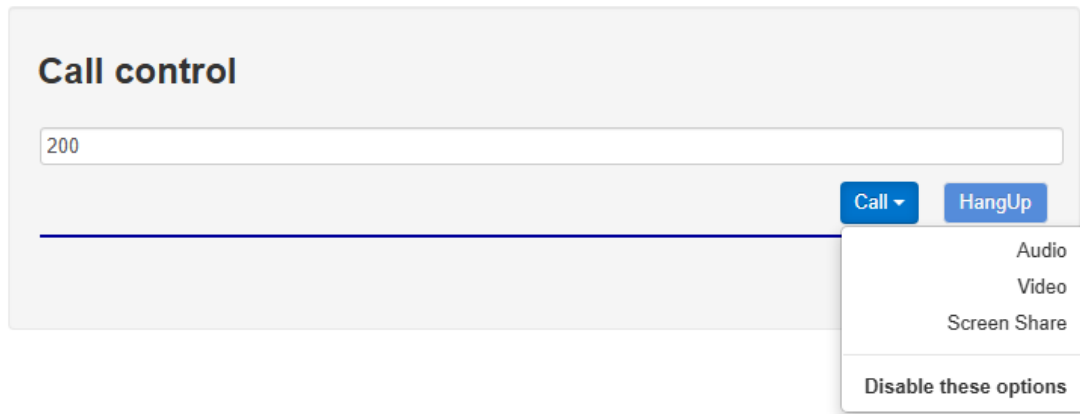


Figure 17 : Call control (*Source : Nous-même*)

Après l'utilisateur peut tester certaine fonctionnalité comme :

V .1.1 Test audio

Objectifs :

- Vérifier la qualité sonore
- Contrôler la stabilité des appels audio

Méthodologie :

- Réalisation d'appels entre deux utilisateurs ou plus
- Mesure de la clarté du son, absence d'écho, niveau de volume
- Test de l'activation/désactivation du microphone

Résultats attendus :

- Communication audio stable et intelligible
- Réduction des bruits de fond et écho minimal grâce aux codecs Opus

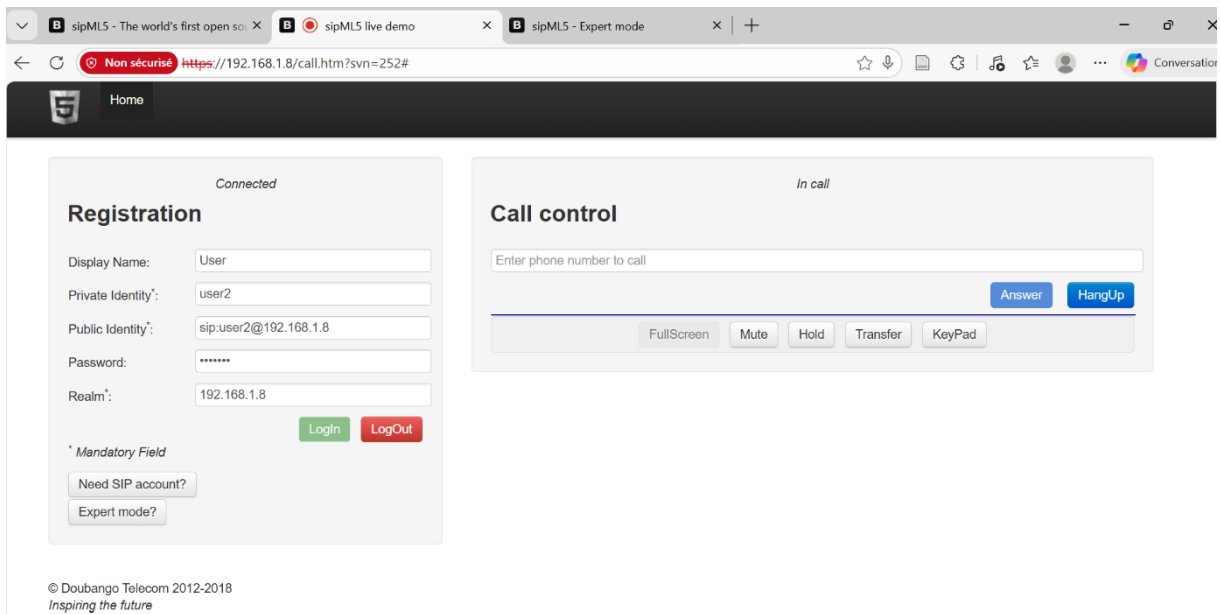


Figure 18 : Test audio (*Source : Nous-même*)

V.1.2 Tests vidéo

Objectifs :

- Vérifier la fluidité et la synchronisation audio/vidéo
- Contrôler l'adaptation de la qualité vidéo selon la bande passante

Méthodologie :

- Échanges vidéo entre deux ou plusieurs utilisateurs
- Simulation de conditions réseau variables (bande passante limitée)
- Test des codecs vidéo VP8 et H.264

Résultats attendus :

- Vidéo fluide avec faible latence
- Ajustement automatique de la résolution pour maintenir la continuité des appels

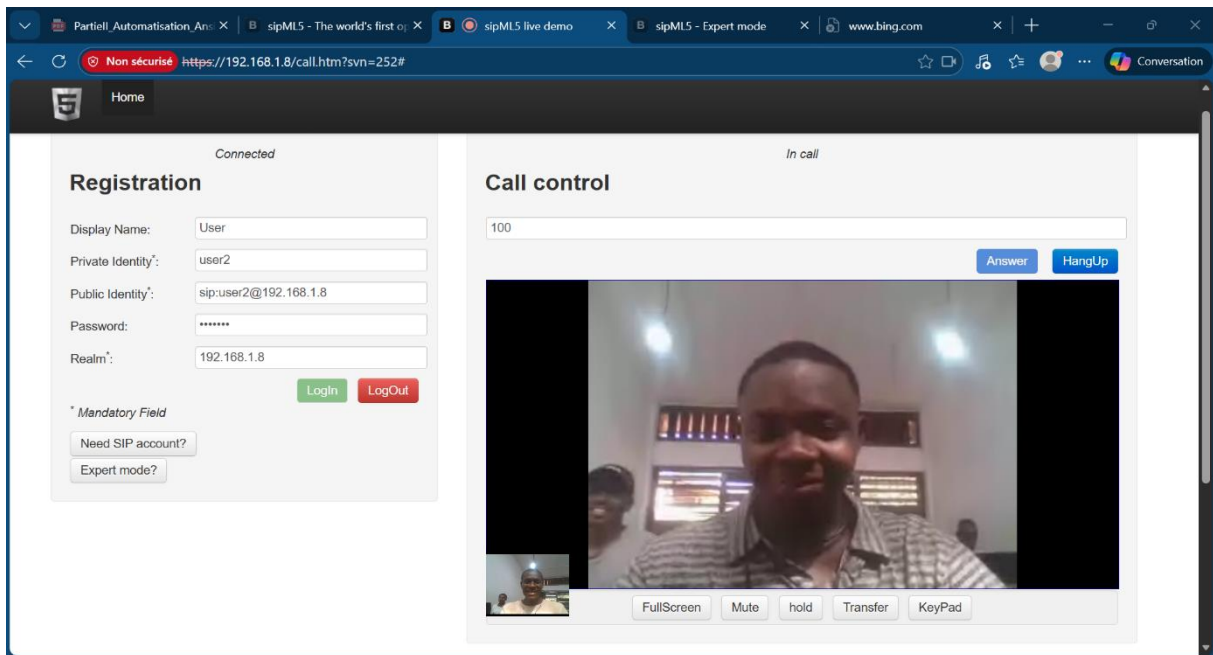


Figure 19 : Tests vidéo (Source : Nous-même)

V.1.3 Tests de partage d'écran

Objectifs :

- Valider la fonctionnalité de partage d'écran ou d'application
- Vérifier la fluidité et la qualité visuelle

Méthodologie :

- Partage complet de l'écran et d'une application spécifique
- Observation du rendu côté récepteur
- Vérification de la possibilité d'arrêter le partage à tout moment
- Résultats attendus :
 - Transmission claire et en temps réel
 - Maintien de la qualité, même lors de sessions multipoints

V.1.4 Tests de messagerie instantanée (chat)

Objectifs :

- Vérifier l'envoi et la réception de messages
- Contrôler la synchronisation avec les appels audio/vidéo

Méthodologie :

- Utilisation du DataChannel WebRTC ou WebSocket
- Envoi de messages simultanés dans différents salons ou appels

Résultats attendus :

- Messages reçus en temps réel
- Aucune perte de données

V.1.5 Tests de gestion des utilisateurs et sessions

Objectifs :

- Vérifier l'authentification et la gestion des comptes
- Contrôler la création et la terminaison correcte des sessions

Méthodologie :

- Connexion/déconnexion des utilisateurs
- Observation des sessions actives sur Asterisk
- Vérification des logs et journaux d'activité

Résultats attendus :

- Authentification sécurisée
- Sessions correctement créées, supervisées et terminées

V.2 Tests de performance

Les tests de performance permettent de mesurer la qualité du service et la scalabilité.

V.2.1 Latence

Mesure du temps entre l'émission et la réception d'un flux audio/vidéo

Objectif : latence inférieure à 150 ms pour une communication fluide

```
C:\Users\NAMTOUA>ping 192.168.1.8 -t

Envoi d'une requête 'Ping' 192.168.1.8 avec 32 octets de données :
Réponse de 192.168.1.8 : octets=32 temps=6 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=6 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=5 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=5 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=5 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=5 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=7 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=5 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=5 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=7 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=7 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=7 ms TTL=64
Réponse de 192.168.1.8 : octets=32 temps=15 ms TTL=64

Statistiques Ping pour 192.168.1.8:
    Paquets : envoyés = 13, reçus = 13, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 5ms, Maximum = 15ms, Moyenne = 6ms
Ctrl+C
^C
C:\Users\NAMTOUA>
```

Figure 20 : Latence (Source : Nous-même)

V.2.2 Bande passante

Mesure de la consommation de la bande passante pour différents types de flux (audio, vidéo, partage d'écran)

Objectif : optimisation des codecs et adaptation dynamique pour limiter la saturation

V.2.3 Charge serveur

Simulation de plusieurs appels simultanés

Observation de l'utilisation CPU et RAM sur le serveur Asterisk

Objectif : maintenir la stabilité même avec un nombre élevé d'utilisateurs

V.3 Tests de sécurité

La sécurité est un élément critique pour protéger les flux et les données des utilisateurs.

V.3.1 Chiffrement des flux

Vérification de l'utilisation de SRTP pour l'audio/vidéo

Vérification de DTLS pour la signalisation WebRTC

V.3.2 Authentification

- Test des mécanismes d'authentification (SIP/WebRTC)
- Vérification de la résistance aux accès non autorisés

V.3.3 Protection contre les attaques réseau

- Simulation et tentative d'intrusion ou d'écoute
- Observation de la robustesse face aux attaques DoS ou MITM

V.4 Méthodologie et outils de test

- Ping et trace-route : mesure de latence et performance réseau
- Test multipoints : utilisation de plusieurs clients WebRTC simultanément
- Rapports de logs Asterisk : suivi des appels et sessions
- Wireshark : capture et analyse des flux RTP/SRTP

V.5 Validation des résultats

Les tests sont validés lorsque :

- ✓ Tous les appels audio et vidéo sont fluides et synchronisés
- ✓ La messagerie instantanée fonctionne correctement
- ✓ La session des utilisateurs est correctement créée et terminées
- ✓ La latence et la bande passante sont optimisées
- ✓ Les flux sont sécurisés et chiffrés

Cette validation confirme l'architecture conçue et déployée est conforme aux besoins identifiés, et peut être utilisée dans un environnement réel.

Ce chapitre nous a permis de tester et valider l'ensemble des fonctionnalités et performances du système WebRTC-Asteris. Les résultats montrent que le système est fiable, performant et sécurisé.

CONCLUSION GENERALE

La conception et la mise en œuvre d'une architecture de visioconférence reposant sur WebRTC et Asterisk ont permis de démontrer la pertinence et la maturité des solutions open source dans le domaine des communications en temps réel. Dans un contexte marqué par la digitalisation croissante et l'exigence de mobilité évoqués dans l'introduction, cette approche technologique se révèle capable de répondre efficacement aux besoins actuels en matière de performance, de sécurité et d'interopérabilité.

L'intégration de WebRTC dédié à la transmission audio-vidéo en temps réel, et d'Asterisk, destiné à la gestion des appels et des protocoles SIP, a permis la construction d'une solution flexible, évolutive et alignée avec les standards modernes de communication. Les différentes étapes du projet analyse des besoins, conception architecturale, déploiement opérationnel, puis validation par des tests ont confirmé la solidité de l'architecture proposée ainsi que sa capacité à offrir une expérience de visioconférence fiable et adaptée à divers contextes professionnels et collaboratifs. Par ailleurs, les résultats obtenus ouvrent des perspectives intéressantes pour des évolutions futures. L'architecture mise en place constitue en effet une base robuste susceptible d'être étendue vers des applications plus avancées, telles que l'intégration de dispositifs IoT, la connexion à des plateformes collaboratives ou encore l'ajout de fonctionnalités intelligentes basées sur l'analyse des flux multimédias. Ces pistes s'inscrivent pleinement dans la logique de transformation numérique des organisations, soulignée dans l'introduction, en offrant une alternative ouverte, durable et technologiquement contrôlable face aux solutions propriétaires.

Ainsi, cette étude confirme qu'une architecture de visioconférence fondée sur WebRTC et Asterisk représente non seulement une solution efficace pour les besoins actuels, mais également un socle prometteur pour le développement d'écosystèmes de communication futurs.

BIBLIOGRAPHIE ET WEBOGRAPHIE

- [1] <http://www.yeast.com>, vidéo/audio conférence using WebRTC consulté le 18 septembre
- [2] https://Laurent.tomczyk.free.fr/serveur_sacprose, consulté le 20 août 2025
- [3] <https://www.onsip.com/voipressources.voip-fondamentals/webrtc> -sip and html5-bref-introduction consulté le 8 octobre 2025
- [4] <https://dspace.Univtemouchent.edu.dz/bitstram>, consulté 11 août 2025
- [5] www.google.com consulté le 21 septembre 2025
- [6] www.google.gemini consulté le 25 septembre 2025
- [7] www.chatgpt.com consulté le 26 septembre 2025
- [8] <https://docs.asterisk.org> Asterisk Documentation consulté le 26 septembre 2025
- [9] <https://github.com> DoubangoTelecom/sipml5: The warlds first HTML SIP client
- [10] <https://www.videosdk.live> SIP.js:simplify webrtc Development for Real Time Apps
- [11] <https://sipjs.com> About Us
- [12] configuration de pjsip.conf compatible WebRTC, consulté le 16 octobre 2025
- [13] INTRODUCTION A MICROSOFT TEAMS CONVERSATION, consulté 6 octobre
- [14] Les protocoles de sécurité WebRTC, consulté 4 octobre 2025
- [15] établissement de la communication sip, consulté le 11 août 2025
- [16] Etude-et-Mise-en-Œuvre-du-WebRTC, consulté le 7 janvier 2025

TABLE DES MATIERES

SOMMAIRE	i
DEDICACE.....	ii
REMERCIEMENTS	iii
RESUME.....	iv
ABSTRACT	v
TABLES DES FIGURES	vi
INTRODUCTION GENERALE.....	1
I.1 Technologies de visioconférence existantes	2
I.1.1 Solutions propriétaires	2
I.1.1.1 Google Meet.....	2
I.1.1.2 Microsoft Teams	3
I.1.1.3 Zoom	4
I.1.2 Solution Open source.....	5
I.1.2.1 BigBlueButton.....	5
I.1.2.2 Présentation du WebRTC	6
I.1.2.2.1 Les fonctionnalités du WebRTC	7
I.1.2.2.2 Les avantages de la technologie WebRTC	7
I.1.2.2.3 Les composantes de la technologie WebRTC	8
I.1.2.2.4 Limites de la technologie WebRTC	8
I.1.2.2.5 Types d'architecture de conférence WebRTC.....	9
I.1.2.3 Présentation d'Asterisk	11
I.1.2.3.1 Architecture	12
I.1.2.3.2 Fonctions principales dans la ToIP	13
I.1.2.3.3 Fonctionnement d'Asterisk	13
CHAPITRE II : ANALYSE DES BESOINS ET SPÉCIFICATIONS	14
II.1 Identification et analyse des besoins fonctionnels.....	14

II.1.1 Besoin de communication audio en temps réel.....	14
II.1.2 Besoin de communication vidéo interactive	15
II.1.3 Besoin de conférence multi-utilisateurs.....	15
II.1.4 Besoin de partage d'écran et de contenus	16
II.1.5 Besoin de communication textuelle (chat).....	16
II.1.6 Besoin de gestion des utilisateurs et des sessions.....	16
II.2 Analyse des besoins non fonctionnels	17
II.2.1 Besoins en matière de sécurité	17
II.2.2 Besoins de performance et de qualité de service	17
II.2.3 Besoins d'évolutivité et de disponibilité.....	17
II.3 Contraintes techniques et environnementales.....	17
II.3.1 Contraintes réseau.....	17
II.3.2 Contraintes matérielles et logicielles	18
CHAPITRE III : CONCEPTION DE L'ARCHITECTURE	19
III.1 Schéma global de l'architecture	19
III.1.1 Composants principaux	19
III.1.2 Schéma conceptuel	20
III.2 Flux de communication.....	20
III.2.1 Signalisation	20
Protocole SIP	21
Protocole H.323	23
Protocole IAX	24
Protocole MGCP.....	24
III.2.2 Transport des médias.....	25
Le protocole RTP.....	25
Le protocole RTCP.....	26
Le protocole TCP.....	26

Le protocole UDP	26
III.3 Intégration des modules	27
III.3.1 Module d'authentification	27
III.3.2 Gestion des sessions	27
III.3.3 NAT traversal (STUN/TURN)	27
ICE (Interactive Connectivity Establishment)	27
STUN (Session Traversal Utilities for NAT)	29
TURN (Traversal Using Relays around NAT)	29
III.3.4 Module de gestion des flux et QoS.....	30
Codecs audios	30
Codecs vidéo.....	31
III.4 Architecture détaillée côté serveur et client	32
CHAPITRE IV : DÉPLOIEMENT ET MISE EN ŒUVRE.....	34
IV.1 Environnement de développement.....	34
IV.1.1 Choix des outils et plateformes	34
IV.1.2 Préparation du serveur.....	34
IV.2 Installation et configuration d'Asterisk.....	34
IV.2.1 Modules nécessaires pour WebRTC	35
IV.2.2 Configuration des comptes utilisateurs	35
IV.2.3 Configuration du serveur HTTP pour WebRTC.....	36
IV.3 Déploiement des clients WebRTC.....	38
SIPML5	38
IV.3.1 Tests de connectivité	39
CHAPITRE V : TESTS ET VALIDATION.....	40
V.1 Tests fonctionnels.....	40
V .1.1 Test audio.....	43
V.1.2 Tests vidéo	44

V .1.3 Tests de partage d'écran	45
V.1.4 Tests de messagerie instantanée (chat)	45
V.1.5 Tests de gestion des utilisateurs et sessions.....	46
V.2 Tests de performance.....	46
V.2.1 Latence.....	46
V.2.2 Bande passante.....	47
V.2.3 Charge serveur	47
V.3 Tests de sécurité	47
V.3.1 Chiffrement des flux.....	47
V.3.2 Authentification	47
V.3.3 Protection contre les attaques réseau	48
V.4 Méthodologie et outils de test	48
V.5 Validation des résultats	48
CONCLUSION GENERALE	49
BIBLIGRAPHIE ET WEBOGRAPHIE	vi